

QuickRuitAI - Product Requirements Document (PRD)

1. Executive Summary

QuickRuitAI is an end-to-end B2B HR-tech platform designed for recruiting and staffing agencies. The platform automates the first-round hiring process through a sequence of customizable modules: Aptitude, Technical, AI Video Interview, and Coding Assessments.

The immediate next phase involves introducing **Recruiter Rian** (an orchestrating conversational AI interviewer), **Dynamic AI Video Interviews**, **Unified Scorecards**, and **Third-Party Integrations** (ATS and Career Pages) to create a seamless, deeply interactive, and secure candidate experience.

2. Feature Comparison: Current State vs. Target State (Milestones)

A. Core Platform & Job Engine

Feature	Current State	Target State / Wants	Priority
Job Configuration	Jobs can be created with selected assessment modules (Aptitude, Technical, Coding, AI Video) & duration. and especially the experience level.	Configuration-driven architecture supporting non-technical roles (Sales, Support) with custom workflows.	Phase 1
Job Publishing	Generates a Magic Link shareable on LinkedIn, Naukri, etc.	Direct Career Page Push: Publish directly to the company's career page. ATS Integration: Sync jobs and candidate statuses with external ATS.	Phase 5

B. Candidate Experience & Assessment Modules

Feature	Current State	Target State / Wants	Priority
Question Banks	Basic schemas exist (AptitudeQuestionBank.js, etc.), but content is limited.	Comprehensive DB architecture for topics, skills, difficulty balancing, and randomization.	Phase 1

Feature	Current State	Target State / Wants	Priority
Aptitude & Tech Rounds	Static modules generating simple scores.	Enforced sequential orchestration; candidate cannot skip.	Phase 2
AI Video Interview	Static questions based on JD + Resume. Output is a detailed scorecard.	Dynamic Interviewing: Adapts questions based on real-time candidate answers. Controlled by interview policy (difficulty, bounds).	Phase 3
Coding Assessment	Structure for DSA and Machine coding exists.	Real-time evaluation with AI hinting (without revealing solutions).	Phase 2

C. Recruiter Rian (AI Orchestrator)

Feature	Current State	Target State / Wants	Priority
Interview Orchestration	Standard UI transitions between rounds.	Recruiter Rian: Conversational AI layer. Introduces the interview, sets expectations, transitions candidate between modules, and gives encouragement/hints.	Phase 3
State Management	Handled loosely on the frontend.	Backend Source of Truth: Backend controls state, Rian controls conversation, Frontend renders active module.	Phase 2

D. Security & Evaluation

Feature	Current State	Target State / Wants	Priority
Proctoring	Tab-switching, face detection, environment checks.	Risk-based engine: Generates a confidence score instead of binary "cheating" labels. Screen sharing integration.	Phase 1
Scorecard & Ranking	Detailed for Video AI; basic scores for others. Ranking exists.	Unified Scorecard: Combines all modules (Aptitude, Tech, Coding, AI Video, Behavioral, Security Risk). Explainable evidence.	Phase 4

3. Structural Integration Plan & Architecture

To integrate the new features seamlessly without breaking the existing flow, the system must adopt a strict separation of concerns:

1. The Interview Engine (State Machine)

- **How it integrates:** Create a dedicated `InterviewSession` model in the backend. This will track the exact module the candidate is currently on, time remaining, and module status.
- **Why:** If a candidate disconnects, they resume exactly where they left off. The backend is the ultimate source of truth, preventing the frontend or AI from hallucinating states.

2. Recruiter Rian (Conversational Orchestrator)

- **How it integrates:** Rian sits between the Interview Engine and the Candidate UI. Rian does *not* decide what round is next; the Interview Engine tells Rian, "Transition to Technical." Rian then generates the contextual dialog: *"Great job on the aptitude round, let's move to technical."*
- **Dynamic Questioning:** For the video interview, feed Rian a strict JSON schema blueprint representing the "Interview Policy". Rian evaluates the candidate's last answer, decides the next topic from the blueprint, and asks a follow-up.

3. Unified Scorecard & Evaluation Engine

- **How it integrates:** Instead of evaluating modules in silos, write an `EvaluationService`. As each module finishes, its raw data is saved to the `InterviewSession`. Once the entire interview concludes, the `EvaluationService` runs an LLM-assisted aggregation over all raw data (tech scores + behavioral analysis + proctoring events) to generate the Unified Scorecard.

4. ATS & Career Page Integration

- **How it integrates:** Introduce a background queue (e.g., BullMQ) and a standard `IntegrationAdapter` pattern. When a job is published, an event is emitted. The adapter translates our job schema to the specific ATS API schema (e.g., Workday, Greenhouse).
-

4. Architectural Feasibility & Honest Feedback

Based on the 3-month timeline, here is my honest assessment:

1. **Feasibility of the 3-Month Vision:** Building *everything* requested in 3 months is highly ambitious and risky. The core AI interviewer (Recruiter Rian) + Unified Scorecards + Dynamic Video AI is a massive undertaking on its own.
2. **What is genuinely difficult:**
 - a. **Dynamic AI Video Interview:** Getting an LLM to reliably conduct a technical interview in real-time, generate audio/video, enforce time constraints, and *never* hallucinate is extremely difficult. Latency will be your biggest enemy here.
 - b. **ATS Integrations:** ATS APIs (Greenhouse, Workday, Lever) are notoriously complex, undocumented, or require expensive partner approvals. Building a universal sync in 3 months is a trap.
3. **What should be postponed (Not MVP):**
 - a. **Screen Sharing:** Screen sharing is very heavy on WebRTC infrastructure. Stick to DOM-based proctoring (tab switches, copy-paste) and camera feeds for the MVP.

- b. **ATS/Career Page Integrations:** Push this to post-MVP. Focus on making the core QuickRuit platform an undeniable success via Magic Links first.
 - c. **Machine-level Coding:** Stick to DSA first. Real-world environment provisioning (Docker/sandboxing) for machine rounds takes significant infrastructure work.
4. **Backend vs AI Control:**
- a. The backend **MUST** control: Timers, state transitions, security event logging, and the final scoring aggregation.
 - b. The AI **MUST NEVER** control: Passing/failing the candidate immediately, time limits, or deciding to skip a mandatory module.
5. **The Ultimate Differentiator:**
- a. Your core differentiator is **Recruiter Rian + The Unified Scorecard**. If you can provide a recruiter with an explainable, deep dive into a candidate's actual capability (with video evidence) rather than just a "Passed/Failed" flag, agencies will love the platform. Focus 80% of your effort here.